

Checkout personalizado en VTEX: flujo completo

A continuación se describe paso a paso cómo implementar un checkout *100% personalizado* usando solo las APIs públicas de VTEX (REST/GraphQL), fuera del flujo visual nativo. Se indican los endpoints, métodos, headers y payload de ejemplo, así como consideraciones de sesión, pagos, seguridad y casos reales.

1. Crear un carrito (orderForm)

La integración comienza creando o recuperando el carrito de compras (orderForm) del usuario. Se usa la ruta del Checkout API **GET**:

```
GET /api/checkout/pub/orderForm?forceNewCart=true
```

- **Descripción:** Solicita el carrito actual o crea uno nuevo si no existe. El parámetro `forceNewCart=true` fuerza la creación de un carrito vacío. ¹.
- **Headers:** Incluir `Accept: application/json`. Normalmente se autentica con cookies (`checkout.vtex.com`) o con AppKey/AppToken en entornos server-to-server.
- **Respuesta:** JSON con `orderFormId` (ID único del carrito) y estructura inicial (items vacío, shippingData nulo, etc.) ¹. Ejemplo mínimo:

```
{
  "orderFormId": "9ceee0fde6db489fbc682a0e2ab13a86",
  "items": [],
  "shippingData": null,
  "clientProfileData": null,
  ...
}
```

- **Orden:** Guardar el `orderFormId` retornado (por ejemplo en cookie/localStorage) para usar en llamadas siguientes ¹ ². El cookie `checkout.vtex.com` suele incluir el `orderFormId` ². Mantenga ese ID para todas las solicitudes subsiguientes.

2. Agregar ítems al carrito

Con el `orderFormId` obtenido, se agregan productos al carrito usando **POST**:

```
POST /api/checkout/pub/orderForm/{orderFormId}/items?
allowedOutdatedData=paymentData
```

- **Descripción:** Añade uno o varios ítems al carrito existente ³. El query `allowedOutdatedData=paymentData` permite ignorar estado obsoleto de información de pago si ya existiera.

- **Headers:** Content-Type: application/json. Cookies o AppKey/AppToken según autenticación.
- **Payload (body):** Objeto JSON con lista orderItems. Cada ítem requiere al menos: id (ID de SKU), quantity, seller (por lo general "1"), price (en centavos) y un index. Ejemplo:

```
{
  "orderItems": [
    {
      "id": "2005",
      "quantity": 3,
      "seller": "1",
      "price": 1099,
      "index": 0
    }
  ]
}
```

Aquí se agregan 3 unidades del SKU 2005 por el vendedor 1 a precio 1099 (suma 32.97). 4.

- **Respuesta:** El orderForm actualizado con los ítems y totales recalculados. Los totalizers reflejarán precios y descuentos. Las promociones configuradas en VTEX se aplican automáticamente en esta llamada si corresponde.

3. Establecer perfil del cliente

Se debe asociar o crear el perfil de cliente en el carrito usando **POST**:

POST /api/checkout/pub/orderForm/{orderFormId}/attachments/clientProfileData

- **Descripción:** Asocia información del cliente (perfil) al carrito. Esto *desenmascara* datos personales en pedidos futuros. Al enviar estos datos, VTEX genera el cookie CheckoutOrderFormOwnership para que solo este cliente acceda a sus datos 5 6.
- **Headers:** Content-Type: application/json.
- **Payload:** JSON con los campos del perfil, como email, nombre, documento, teléfono, etc. Ejemplo:

```
{
  "email": "cliente@ejemplo.com",
  "firstName": "Juan",
  "lastName": "Pérez",
  "documentType": "cpf",
  "document": "12345678900",
  "phone": "+55999998888",
  "corporateName": "",
  "tradeName": "",
  "corporateDocument": "",
  "stateInscription": "",
  "corporatePhone": ""
}
```

En este ejemplo se envía un perfil de persona física. Se deben respetar los formatos configurados en VTEX. ⁶ .

- **Respuesta:** El carrito actualizado con `clientProfileData` asignado. VTEX establece el cookie `CheckoutOrderFormOwnership` con este contenido cifrado ⁷ . Con esa cookie, posteriores GET al orderForm devolverán los datos del cliente completos; sin ella quedarían enmascarados (prevención de exposición no autorizada) ⁵ .

4. Configurar dirección y logística

El siguiente paso es añadir la dirección de entrega (o retiro) y seleccionar la logística. Se hace con **POST**:

```
POST /api/checkout/pub/orderForm/{orderFormId}/attachments/shippingData
```

- **Descripción:** Añade información de dirección(es) y opciones de envío al carrito. Esto incluye la dirección postal y la entrega seleccionada por cada ítem o conjunto de ítems ⁸ ⁹ .
- **Headers:** `Content-Type: application/json` .
- **Payload:** JSON con campos clave:
- `selectedAddresses` : lista de direcciones usadas. Cada dirección con tipo (`"residential"`), receptor, código postal, ciudad, estado, calle, número, complementos, coordenadas, etc. ⁸ . Por ejemplo:

```
{
  "selectedAddresses": [
    {
      "addressType": "residential",
      "receiverName": "María López",
      "addressId": "c3701fc4c61b4d1b91f67e81415db44d",
      "postalCode": "12345-000",
      "city": "Madrid",
      "state": "MD",
      "country": "ESP",
      "street": "Calle Falsa",
      "number": "100",
      "neighborhood": "Centro",
      "complement": "",
      "reference": "Edificio XYZ",
      "geoCoordinates": [-3.703790, 40.416775]
    }
  ],
  "logisticsInfo": [
    {
      "itemIndex": 0,
      "selectedSla": "Express",
      "selectedDeliveryChannel": "delivery"
    }
  ]
}
```

En este ejemplo se define una sola dirección de tipo residencial y se selecciona el servicio de entrega "Express" para el ítem en `itemIndex: 0`. Los valores (`addressId`, SLAs disponibles, etc.) deben provenir de opciones válidas (previamente consultadas con el carrito o configuraciones de la tienda) ⁸ ⁹.

- **Respuesta:** El carrito con `shippingData` lleno, incluyendo `deliveryOptions` y `selectedAddresses`. El cookie `CheckoutOrderFormOwnership` se actualiza también tras esta llamada ¹⁰. A partir de aquí, todos los datos del cliente y dirección aparecen sin mascarar (poner atención a la cookie ownership ¹¹).

5. Inyectar información de pago

Con el carrito armado (items, perfil, shipping), hay que añadir los datos de pago. Esto se hace con **POST**:

```
POST /api/checkout/pub/orderForm/{orderFormId}/attachments/paymentData
```

- **Descripción:** Envía la información de pago (método, importe, token, cuotas, etc.) asociada al carrito. El cuerpo es un arreglo `payments` o un objeto `paymentData` con los datos necesarios para el pasarela. Según VTEX, se basa en los campos de `orderForm.paymentData` ¹².
- **Headers:** `Content-Type: application/json`.
- **Payload:** Un objeto con, por ejemplo, `payments: [ {...} ]`. Cada elemento incluye:
- `paymentSystem`: ID del medio de pago autorizado (por ejemplo, 2 = Visa/Adyen) ¹².
- `installments`: cantidad de cuotas.
- `currencyCode`, `value` (importe total en centavos) y `referenceValue`.
- `tokenId`: token de la tarjeta o transacción si aplica (ver abajo).
- Otros campos según medio (campos extra en `fields` o `transaction`). Ejemplo simplificado:

```
{
  "payments": [
    {
      "paymentSystem": 2,
      "installments": 1,
      "currencyCode": "EUR",
      "value": 10997,
      "referenceValue": 10997,
      "fields": {},
      "transaction": {
        "id": "72E84719BDF14B2FB170B38AD12598C9",
        "merchantName": "MID_TIENDA"
      }
    }
  ]
}
```

Aquí se indica pago con el sistema 2 (por ejemplo Adyen Visa), una cuota y total 109.97€. El campo `transaction.id` puede ser un identificador interno o referencia. Si el medio requiere token (p.ej. pago con tarjeta), se llenaría `tokenId` con el token obtenido del PSP, y se indicaría `accountId` con la tarjeta guardada si aplica. ¹².

- **Tokenización:** El `tokenId` debe obtenerse previamente vía el método del proveedor de pago (por ejemplo, mediante un SDK de MercadoPago, Adyen o Fiserv) y corresponde al token de la

tarjeta. VTEX no genera el token; simplemente lo recibe y envía a la pasarela. (En la documentación pública no hay detalles concretos de generación de token por API; esto se gestiona del lado del PSP o mediante endpoints de VTEX Payment Gateway).

- **Respuesta:** El carrito con `paymentData` actualizado. Si el pago se procesa vía pasarela externa, se recibe en respuesta el ID de transacción de VTEX (`orderId`, `transactionId`) que usará el siguiente paso).

6. Finalizar el pedido

Con todo el carrito y pago configurado, se crea la orden en VTEX. Hay dos variantes, la más común es usar el "Place Order from existing cart":

- **Endpoint (Place order):**

```
POST /api/checkout/pub/orderForm/{orderId}/transaction
```

Esta ruta genera el pedido (order) basado en el carrito existente ¹³. En el body se puede repetir (`orderId`) o no es necesario. Tras esta llamada VTEX asigna un `orderId` y un `transactionId`. La respuesta es 200 con el ID del pedido.

- **Ejemplo:**

```
curl -X POST "https://{cuenta}.vtexcommercestable.com.br/api/checkout/pub/orderForm/{orderId}/transaction" \
-H "Content-Type: application/json"
```

(No se necesita body adicional si todo el `orderForm` ya está cargado.)

- **Webhook / Llamada final:** Después de la creación, se debe *procesar* el pago e informar a VTEX. Esto se hace invocando **POST** al endpoint de proceso (gateway callback) usando el `orderGroup` (que coincide con el `orderId` o con el `orderGroup` retornado). En la práctica:

```
POST /api/checkout/pub/gatewayCallback/{orderGroup}
```

donde `{orderGroup}` es el identificador del grupo de orden (usualmente el mismo `orderId`) ¹⁴. Este paso confirma a VTEX que el pago fue exitoso. Si la pasarela lo permite, debe hacerse dentro de 5 minutos tras la creación (de lo contrario VTEX cancelará el pedido) ¹⁵.

- **Ejemplo (cURL):**

```
curl -X POST "https://{cuenta}.vtexcommercestable.com.br/api/checkout/pub/gatewayCallback/9ceee0fde6db489fbc682a0e2ab13a86" \
-H "X-VTEX-API-AppKey: {appKey}" \
-H "X-VTEX-API-AppToken: {appToken}"
```

Se espera un `204 No Content` si el pago fue aprobado ¹⁴.

- **Respuesta final:** Al concluir, la orden queda en sistema. VTEX devolverá un `orderId` que podrá usarse con la OMS (Order Management System) para consultas futuras. Si ocurre un error (p.ej. código 500), el flujo debe manejarlos.

7. Sesiones, cookies y manejo de `orderFormId`

- **Session:** Al usar un checkout personalizado, asegúrese de preservar la sesión del usuario (cookies de VTEX). El cookie clave es `checkout.vtex.com`, que contiene el `orderFormId`. Si el sitio es headless, la app debe leer/almacenar este cookie localmente o en cada petición. ².
- **CheckoutOrderFormOwnership:** VTEX genera una cookie `CheckoutOrderFormOwnership` al agregar perfil/dirección ¹¹. Esta protege datos PII del cliente. Su integración debe reenviarla en posteriores solicitudes si quiere ver nombres, emails, etc. Si no la incluye, las API devolverán datos «enmascarados».
- **Reuso del carrito:** Guarde el `orderFormId` para que, si el usuario abandona el sitio y vuelve, pueda recuperar el carrito previo con `GET /orderForm/{orderFormId}`. Si cada vez se llama sin ID, se crearán muchos carritos nuevos ².
- **Headers VTEX:** Para llamadas server-to-server, incluya siempre `X-VTEX-API-AppKey` y `X-VTEX-API-AppToken` con credenciales de la cuenta VTEX. Para front-end, normalmente bastarán las cookies de sesión y CORS habilitado.

8. Integración de pasarelas de pago (MercadoPago, Adyen, Fiserv, etc.)

- **Payment Systems autorizados:** En el **Admin de VTEX** debe configurarse el o los métodos de pago (Payment Providers) que se van a usar (MercadoPagoV2, Fiserv, Adyen, etc.). Cada uno recibe un ID numérico (`paymentSystem`) que luego se usa en la API. Por ejemplo, Adyen Visa suele ser `2` ¹², MercadoPago tiene su propio ID, etc.
- **Tokenización:** Casi todos los medios de pago requieren tokenizar la tarjeta de crédito/débito. Esto se hace fuera de las APIs de Checkout: por ejemplo, usando el SDK/JS de MercadoPago o Adyen para obtener un `tokenId` al procesar los datos de la tarjeta en el navegador. Ese `tokenId` luego se envía en el campo `tokenId` dentro de `paymentData` al finalizar el pedido. VTEX **no** provee un endpoint REST para generar el token de la tarjeta; se asume que lo obtiene el front-end del PSP.
- **Ejemplo de flujo:** Un cliente ingresa datos de tarjeta en un formulario integrado con MercadoPago. MercadoPago devuelve un token JS. Usted incluye ese `token` en el JSON de pago al llamar a VTEX. VTEX envía ese token al PSP configurado en la plataforma para autorizar el pago.
- **Campos especiales:** Dependiendo del PSP, puede haber campos extra en `fields` (por ejemplo, datos de tarjeta guardada, `cvv`, etc.) o el `transaction.id` para guiar la reconciliación. Siempre valide que los `value` (monto) coincidan exactamente con el carrito. ¹⁶.
- **Pagos guardados:** VTEX permite tarjetas guardadas (Vault). Para usarlas, primero se obtienen las cuentas disponibles usando `GET /api/checkout/pub/profiles?email=user@mail` ¹⁷, luego en `paymentData` se coloca el `accountId` de la tarjeta guardada y se pide el CVV. El proceso de token es similar.

Nota: La documentación oficial de VTEX no detalla cada medio externo, pero el esquema general es: usar los medios configurados en Admin, capturar `tokenId` con cada PSP y enviarlo en el `paymentData` del checkout.

9. Casos reales de checkout headless

Algunas tiendas han implementado exitosamente checkouts 100% personalizados usando APIs de VTEX. Un ejemplo destacado es **Cobasi** (cadena de tiendas de mascotas en Brasil); en 2019 su equipo

interno (Cobasi Labs) desarrolló un checkout propio en VTEX usando únicamente las APIs (sin usar el checkout UI prediseñado) . Reportaron *20% de aumento en tasa de conversión* gracias al mejor desempeño y UX ¹⁸ ¹⁹ . Otro caso es **Grupo Soma** (moda Brasil) que creó front-ends headless para varios de sus sitios. Estos casos confirman que VTEX permite desacoplar el checkout visual mientras se apoya en el motor de órdenes backend.

10. Validación de orden y stock

Tras completar el pedido, conviene verificar que todo se procesó correctamente:

- **Verificar orden:** Use la API de Ordenes de VTEX (OMS) para confirmar. Por ejemplo:

```
GET /api/oms/pvt/orders/{orderId}
```

Requiere credenciales (AppKey/Token con permisos OMS). Retorna detalles del pedido (estado, items, totales, etc.). Esto permite comprobar que el pedido fue creado con el monto y ítems esperados ²⁰ .

- **Impacto en stock:** Cuando se crea el pedido en VTEX, el sistema reserva/descuenta el inventario automáticamente. En general, no es necesario llamar a otra API para restar stock tras la compra. (Si luego se modifica el pedido, VTEX sí requiere actualizar inventario manualmente ²¹ .) Para asegurarse, se puede usar la **Inventory API** de VTEX o la **Logistics API**. Por ejemplo, consultar stock actual de un SKU:

```
GET /api/logistics/pvt/inventory/skus/{skuId}/warehouses
```

Devolverá niveles de inventario. Tras la venta, este número debe reducirse en la cantidad vendida.

- **Promociones y cupones:** Si el carrito incluyó promociones o cupones, éstas ya se aplicaron durante la llamada de add items (VTEX recalcula automáticamente descuentos). Verifique que `orderForm.totalizers` o el resultado final de la orden reflejen los descuentos acordados.
- **Errores:** Si algo falla (por ejemplo, stock insuficiente detectado en `POST /items` o error de pago), revise la respuesta de cada API. VTEX suele devolver JSON con código y mensaje de error para diagnóstico.

11. Seguridad (reCAPTCHA, antifraude, validación)

- **reCAPTCHA:** VTEX soporta reCAPTCHA (v2/v3) para bloquear órdenes de bots. En un checkout personalizado debe considerarse: si la cuenta tiene reCAPTCHA *activo*, cualquier intento de crear orden vía API requerirá la validación del usuario. En la práctica, debe incluir un widget de reCAPTCHA en el flujo. De lo contrario, las API de orden devolverán error. Los docs advierten: "*Si reCAPTCHA está activo, un checkout headless puede bloquear la finalización del pedido, ya que no es posible generar la clave de validación solo vía API; solo con el widget se consigue el token*" ²² . En resumen, o bien desactive reCAPTCHA para ese canal, o implemente la comprobación de usuario real (un desafío).
- **Validación de cliente:** Además del reCAPTCHA, VTEX genera cookies de seguridad. El `CheckoutOrderFormOwnership` mencionado previene que otro usuario vea datos PII. Asimismo, es buena práctica validar (en su código) que el cliente esté autenticado o autorizado (p.ej. comparar `userId`).

- **Antifraude:** Si en VTEX Admin tiene configurado un proveedor antifraude (p.ej. CyberSource, Shield), VTEX evaluará la orden automáticamente durante su creación. Su integración no requiere pasos extra aparte de pasar el pago. Sólo notar que, si la orden es marcada como fraudulenta, VTEX la bloqueará. Debe monitorear su estado via OMS (y quizás retener envío hasta revalidación).
- **Comunicación segura:** Use siempre HTTPS. Las credenciales AppKey/Token o cookies deben enviarse en cabeceras/HTTPS. Evite exponer la `orderFormId` públicamente.

12. Riesgos y soporte oficial

- **Soporte VTEX:** VTEX **no garantiza soporte** para soluciones completamente customizadas fuera de su flujo nativo. De hecho, advierte que *"no respalda el uso de scripts personalizados y no se responsabiliza por daños causados por ellos"* ²³. Modificaciones avanzadas de checkout pueden quebrar con actualizaciones de plataforma o caer fuera de prácticas recomendadas.
- **Mantenimiento futuro:** Con un checkout 100% personalizado, es posible que futuras mejoras de VTEX (nuevas validaciones, cambios de API) requieran ajustes propios, porque no se usa la solución oficial. Evalúe esto: la ventaja de control se sacrifica con la obligación de mantener la integración manualmente.
- **Cumplimiento y PCI:** Si almacena o procesa datos de tarjeta (incluso via token), asegúrese de cumplir PCI-DSS y usar métodos aprobados. Idealmente el formulario de pago debe enviarse directo al PSP (o usar VTEX JS Library) para minimizar exposición de datos sensibles.
- **Reglas VTEX:** Algunos procesos obligan a pasar por VTEX. Por ejemplo, **el pedido final (checkout)** debe realizarse vía sus endpoints para actualizar orden, stock y promociones correctamente. No es posible crear la orden "externamente" sin usar VTEX, porque entonces no se sincronizaría en el OMS y el inventario no se gestionaría dentro de la plataforma. De modo similar, cualquier cálculo de tarifas o impuestos debe hacerse con las APIs de VTEX (o a través de su sistema), para no desalinear datos.

13. Partes que deben procesarse en VTEX

A fin de cuentas, aún usando un checkout custom, varios componentes clave *deben* pasar por VTEX: -

Orden y stock: La creación del pedido final se envía a VTEX (punto 6). VTEX es quien realmente confirma la orden y descuenta el inventario. No se puede omitir este paso sin perder coherencia.

- **Antifraude y pagos:** Si la tienda usa el sistema de pagos/antifraude de VTEX (por ejemplo, pasarelas configuradas en Admin), estos flujos se integran con los endpoints descritos. Si por ejemplo procede a capturar pagos fuera de VTEX, la orden quedaría sin marcar como pagada en VTEX.

- **Promociones y precios:** Los precios con descuentos, reglas de promoción, cupones, etc., son gestionados por VTEX. Al agregar ítems al carrito vía API, se aplica el motor de promociones de VTEX. Evite replicar esas reglas externamente.

En resumen, usted provee la experiencia de usuario y recopila datos (frontend), pero la "lógica dura" de checkout (carrito, orden, impuestos, stock) permanece en VTEX mediante sus APIs ¹ ¹³.

Fuentes: Documentación oficial de VTEX (Checkout API, guías de integración headless) ¹ ⁴ ⁶ ⁸ ¹² ¹³ ¹⁴ ¹⁸ ¹⁹ ²² ²³. Se asume conocimiento técnico de las APIs de VTEX. Códigos de datos específicos de pasarelas de pago deben obtenerse de la documentación del proveedor correspondiente.

1 3 4 Checkout API | Documentation | Postman API Network

<https://www.postman.com/bbozon/vtex/documentation/uee32bw/checkout-api>

2 Consultar los datos del carrito - Español - VTEX Community

<https://community.vtex.com/t/consultar-los-datos-del-carrito/41712>

5 6 7 8 9 10 11 12 13 16 17 Headless cart and checkout

<https://developers.vtex.com/docs/guides/headless-cart-and-checkout>

14 15 20 Creating a regular order with the Checkout API

<https://developers.vtex.com/docs/guides/creating-a-regular-order-with-the-checkout-api>

18 19 El surgimiento de los laboratorios de innovación del sector minorista - España

<https://vtex.com/es-es/blog/el-surgimiento-de-los-laboratorios-de-innovacion-del-sector-minorista/>

21 Order modifications

<https://developers.vtex.com/docs/guides/order-modifications>

22 reCAPTCHA

<https://developers.vtex.com/docs/guides/recaptcha>

23 Checkout customization guide

<https://developers.vtex.com/docs/guides/checkout-customization-guide>